Numpy

Irene Siragusa, PhD

CHILAB
Università degli Studi di Palermo

Numpy

- **NumPy** is the algebric computing library that implements vectors and arrays of numeric type and operations on them.
- The n-dimensional array (**ndarray**) is structured as an object of a certain type and its own layout in memory.
- The ndarray is characterized by its **shape** that is the tuple of its dimensions.

Numpy

- Faster vectorial operations compared with loops
- Numerical basis for SciPy, Pandas, scikit-learn libraries

```
# in terminal with the enviromment activated
pip install numpy
# in notebook cell
! pip install numpy
# import .ipynb or .py file
import numpy as np
```

Numpy – creation

```
# from list
a = np.array([1, 2, 3, 4, 5])
print(a)          # [1 2 3 4 5]

# from tuple
b = np.array((0.5, 1.5, 2.5))
print(b.dtype) # float64

# explicit dtype
c = np.array([1, 2, 3], dtype=np.float32)
```

Numpy – generation functions

```
np.zeros(5)      # [0. 0. 0. 0. 0.]

np.ones(4)       # [1. 1. 1. 1.]

np.full(3, 7)    # [7 7 7]
```

Numpy – numerical sequences

```
# arange: similar to range()
np.arange(5)                # [0 1 2 3 4]
np.arange(0, 10, 2)         # [0 2 4 6 8]

# linspace: n equidistant points
np.linspace(0, 1, 5)        # [0. 0.25 0.5 0.75 1. ]

# logspace: n equidistant points in log scale
np.logspace(0, 2, 3)        # [ 1. 10. 100.]
```

Numpy – from other iterables

```
# from generator
gen = (x**2 for x in range(5))
np.fromiter(gen, dtype=int)          # [0 1 4 9 16]

# da string / file
np.fromstring('1 2 3', sep=' ')      # [1. 2. 3.]
```

Numpy – Matrices

```
# list of lists → matrix 3×4
M = np.array([ [1, 2, 3, 4],
               [5, 6, 7, 8],
               [9,10,11,12]])
print(M.shape)          # (3, 4)
print(M.ndim)           # 2
print(M.size)           # 12

np.zeros((3, 4))        # all zeros
np.ones((2, 5))         # all ones
np.eye(4)               # identity matrix
np.diag([1, 2, 3])      # diagonal matrix
```

Numpy – element access

```
M = np.array([
                [1,2,3],
                [4,5,6],
                [7,8,9]])

M[0, 1]          # row 0, col 1 → 2
M[2, :]          # 3rd row      → [7 8 9]
M[:, 1]          # 2nd col      → [2 5 8]
M[0:2, 1:3]      # sub-matrix   → [[2 3],[5 6]]
```

Numpy – operations

```
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])

A + B                    # elment-wise sum
A * B                    # elment-wise product
A @ B                    # matrix product
np.dot(A, B)             # same as A @ B

A.sum()                  # all element sum
A.sum(axis=0)            # col sum → [4 6]
A.sum(axis=1)            # row sum → [3 7]
A.mean(), A.std(), A.min(), A.max()
```

Numpy – shape and dtype

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
a.shape          # (2, 3) → 2 rows, 3 cols
a.dtype          # int64
a.ndim           # 2 → axis number
a.size           # 6 → total number of elements
a.itemsize       # 8 → byte per element
# type change
a.astype(np.float64) # copy with dtype float64
a.astype(np.complex128)
```

Numpy – reshape

```
a = np.arange(12)          # [0..11] shape (12,)
b = a.reshape(3, 4)        # shape (3, 4)
c = a.reshape(2, 2, 3)     # shape (2, 2, 3)

# -1: numpy automatically calculate the final shape
d = a.reshape(4, -1)       # shape (4, 3)
e = a.reshape(-1, 6)       # shape (2, 6)

# flatten: 1d version of the input as a copy
b.flatten() # shape (12,)

# ravel: as flatten but avoids copy if possible
b.ravel() # shape (12,)
```

Numpy – View vs copy

reshape() returns a view, not a copy, if you modify a view, the original array is modified too.

use .copy() to obtain an indepent array.

Numpy – transpose

```
M = np.array([[1, 2, 3],
              [4, 5, 6]]) # shape (2, 3)

# Transpose
T = M.T
T = M.transpose()
T = np.transpose(M) # shape (3, 2)
```

Numpy – axis add/remove

```
v = np.array([1, 2, 3])          # shape (3,)

v[:, np.newaxis]                 # shape (3, 1) col

v[np.newaxis, :]                 # shape (1, 3) row

np.expand_dims(v, axis=0)        # shape (1, 3)

np.squeeze(v[:, np.newaxis])     # shape (3,)
```

Numpy – 1d slicing

```
a = np.arange(10)     # [0 1 2 3 4 5 6 7 8 9]
a[2:5]                # [2 3 4]
a[:4]                 # [0 1 2 3]
a[6:]                 # [6 7 8 9]
a[::2]                # [0 2 4 6 8] step 2
a[::-1]               # [9 8 7 6 5 4 3 2 1 0] inverse
a[1:8:3]              # [1 4 7]
```

Numpy – 2d slicing

```
M = np.arange(16).reshape(4, 4)
M[1, :]          # 2nd full row
M[:, 2]          # 3rd full col
M[0:2, 1:3]      # sub-matrix row 0-1, col 1-2
M[::2, ::2]      # even row and cols

# assign values in slicing + in-place modify
M[0, :] = 0      # zeros on the 1st row
M[:, 0] = -1     # -1 on the 1st col
```

Numpy – boolean indexing

```
a = np.array([3, -1, 4, -1, 5, -9, 2, 6])
# create a mask
mask = a > 0
# mask: [ True False True False True False True True]
# select elements
a[mask]    # [3 4 5 2 6]
a[a > 0]   # compat analog form
# modify selected elements
a[a < 0] = 0 # put a zero negative values
# a: [3 0 4 0 5 0 2 6]
```

Numpy – masks

```
a = np.arange(-3, 4) # [-3 -2 -1 0 1 2 3]

# np.where(cond, val_true, val_false)
np.where(a >= 0, a, -a) # [3, 2, 1, 0, 1, 2, 3])

# combinations of multiple conditions
(a > -2) & (a < 2)   # [-1 0 1]
(a < -1) | (a > 1)   # [-3 -2 2 3]
~(a == 0)            # all except 0

# index of true elements
np.where(a > 0)      # (array([4, 5, 6]),)
```

Numpy – broadcasting

- Broadcasting allows operations between arrays of different shapes without creating explicit copies of the data

`(3,) + scalar → (3,)` adds the scalar value to each element

`(3, 4) + (4,) → (3, 4)` row is added to each row of the matrix

`(3, 1) + (1, 4) → (3, 4)` outer product, col vector x row vector

`(2,3,4) + (4) → (2,3,4)` sum over last axis

Numpy – operations

```
a = np.array([1., 2., 3., 4.])
b = np.array([10., 20., 30., 40.])


# element-wise operations
a + b      # [11. 22. 33. 44.]
a * b      # [10. 40. 90. 160.]
b / a      # [10. 10. 10. 10.]
a ** 2     # [ 1. 4. 9. 16.] scalar broadcasting
```

Numpy – operations

```
# ufunc: universal functions applied element-wise


np.sqrt(a)      # [1. 1.414 1.732 2. ]
np.exp(a)       # [e e^2 e^3 e^4]
np.sin(np.pi * a)
np.log(b)       # [2.30 3.00 3.40 3.69]
```

Numpy – eigenvalues and eigenvectors

```
A = np.array(  [[4., 2.],
               [1., 3.]])

eigenvalues, eigenvectors = np.linalg.eig(A)
print(eigenvalues) # [5. 2.]
print(eigenvectors) # [[ 0.894 -0.707] [ 0.447 0.707]]
```

Numpy – utility functions

```
np.linalg.det(A)            # determinant
np.linalg.inv(A)            # inverse matrix
np.linalg.matrix_rank(A)    # rank

# norms
np.linalg.norm(b)           # L2 norm (Euclidean dstance)
np.linalg.norm(b, ord=1)    # L1 norm

# decompositions
U, s, Vt = np.linalg.svd(A) # SVD
```

Numpy

- Documentation: https://numpy.org/doc/stable/
- Quickstart tutorial: https://numpy.org/doc/stable/user/quickstart.html
- Beginners Guide: https://numpy.org/doc/stable/user/absolute_beginners.html
- numpy.linalg reference: https://numpy.org/doc/stable/reference/routines.linalg.html
- 100 numpy Exercises: https://github.com/rougier/numpy-100